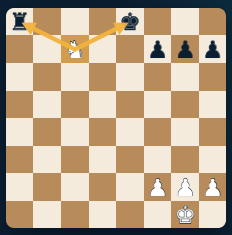
TURNING A CHESS ENGINE INTO SCIENCE AI

Chess AI is now solving science problems.

The search that lets a computer beat chess champions now helps scientists at **Brookhaven National Laboratory** decide which measurement to take next. In two weeks, you and a partner build that kind of AI yourselves in Python, starting with a chess engine named **Remus**, and then turn its thinking on a science puzzle.

10 days, 8:30 to 4:30 | Ages about 12 to 14 | No coding experience needed

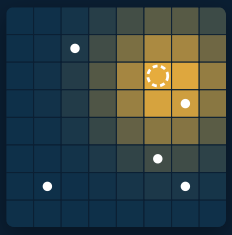


ON THE BOARD

Which move next?

Remus looks ahead and finds the knight fork.

SAME SEARCH: LOOK AHEAD, SCORE, CHOOSE



IN THE LAB

Which measurement next?

An AI picks the spot most likely to reveal the hidden peak.

What you and your partner will build

1

An engine that knows every rule

Castling, en passant, promoting to a knight, check, checkmate, stalemate, and draws by repetition.

2

A player inside a real chess app

On Day 5 you play your own engine in Lucas Chess, a free app chess players use. Later you watch it take on Stockfish.

3

A thinker that looks ahead

Remus searches thousands of positions, scores them, and tells you why it chose its move. The finished class engine rates about 1520 Elo.

4

A power-up, battle-tested

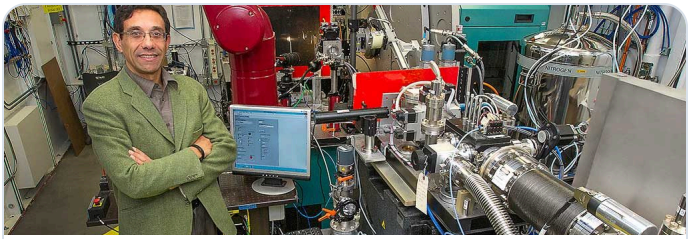
On Day 9 your team adds one power-up to Remus, then carries it on a USB drive to battle the other teams' engines.

FUN FACT YOU WILL PROVE

Chess explodes fast.

Count every possible game from the starting position. Your code has to match these exact numbers, and on Day 6 it will.

After 1 move	20
After 2 moves	400
After 3 moves	8,902
After 4 moves	197,281



YOUR SCIENCE CHALLENGE

Find the hidden peak.

On Day 10 you program an AI that tracks down a hidden hot spot in as few measurements as possible. Then you aim your chess search at a real beamline question: which crystals are in a powder? Your code searches every mix of crystals for the one whose X-ray fingerprints match the measured pattern, the way scientists at light sources like Brookhaven's NSLS-II name new materials.

NSLS-II beamline. Photo: Brookhaven National Laboratory

Not a chess-strength contest
A beginner who knows the moves is ready. The engine does the hard calculating.

Not a typing class
You and your partner trade turns: one types, one checks and steers.

Not a black box
Every line of Remus is plain Python you can read and explain.

WHAT THE TWO WEEKS LOOK LIKE

From your first line of code to your own AI in ten days.

Every day your engine can do something it could not do the day before: make a move, spot a check, play in a real chess app, see moves ahead. On Day 9 each team gives Remus a power-up and measures it. Then you aim your AI at a science problem modeled on Brookhaven's labs: choosing the measurement that reveals what a sample is hiding. By the end you can explain every line of an AI you built yourself.

Every day TEN WEEKDAYS IN A ROW

8:30 to 10:00	10:00	10:30 to 12:00	12:00 to 1:00	1:00 to 2:30	2:30	3:00 to 4:30
Period 1	Break	Period 2	Lunch	Period 3	Break	Period 4

How a period runs: a hands-on puzzle with paper and cards, a small Python lab to practice the idea, then you write a real Remus function and run its tests until they pass.

Ten days, one engine EACH DAY ADDS A PIECE

DAY 1
Setup and the board
Your laptop, VS Code, and Python. Draw a chessboard in code and see it as a picture.

DAY 2
Functions and records
Teach Remus to make a move and take it back exactly.

DAY 3
Loops that walk the board
Knights, kings, rooks, bishops, and pawns, shown as arrows on the board.

DAY 4
Legal moves
Is the king attacked? Throw away every move that leaves it in check.

DAY 5
Special moves, in a real app
Castling, en passant, and promotion. Then play your Remus in Lucas Chess.

DAY 6
Counting the tree
Count every possible game, match the official numbers, and read any position.

DAY 7
A whole game
Checkmate, draws, chess notation, and scoring a position.

DAY 8
Searching ahead
Look several moves ahead, then cut the work with alpha-beta.

DAY 9
Power-ups and battles
Each team adds one power-up to Remus, then battles the other teams to find the strongest.

DAY 10
Solve a science problem using chess AI
Find a hidden peak, name the crystals in an X-ray pattern, and share your results.

What you take home

1. **Your engine on a USB drive**, with every day's code that you and your partner wrote.
2. **Lucas Chess at home**, free from lucaschess.pythonanywhere.com for Windows or Linux (no Mac version). Add your Remus as an engine.
3. **Your match and battle games**, saved so you can replay every move.
4. **Your team name in the credits** of the class engine for each function you wrote.
5. **Real Python skills**: functions, loops, lists, recursion, testing, and version control.

For parents

Does my child need a laptop?
No. Each pair uses a class laptop with everything already installed.

Will my child need online accounts?
No. There are no sign-ups of any kind. All work stays on the class laptops and goes home on USB.

What if my child has never coded?
That is expected. Python is taught from the first line, with labs before every engine task.

Who is in the room?
Twenty students in ten pairs, with an instructor and an assistant, so help is never far away.

You are ready if you

- ✔ know how each chess piece moves
- ✔ like solving puzzles with a partner
- ✔ are curious how computers think
- ✔ want to learn about AI